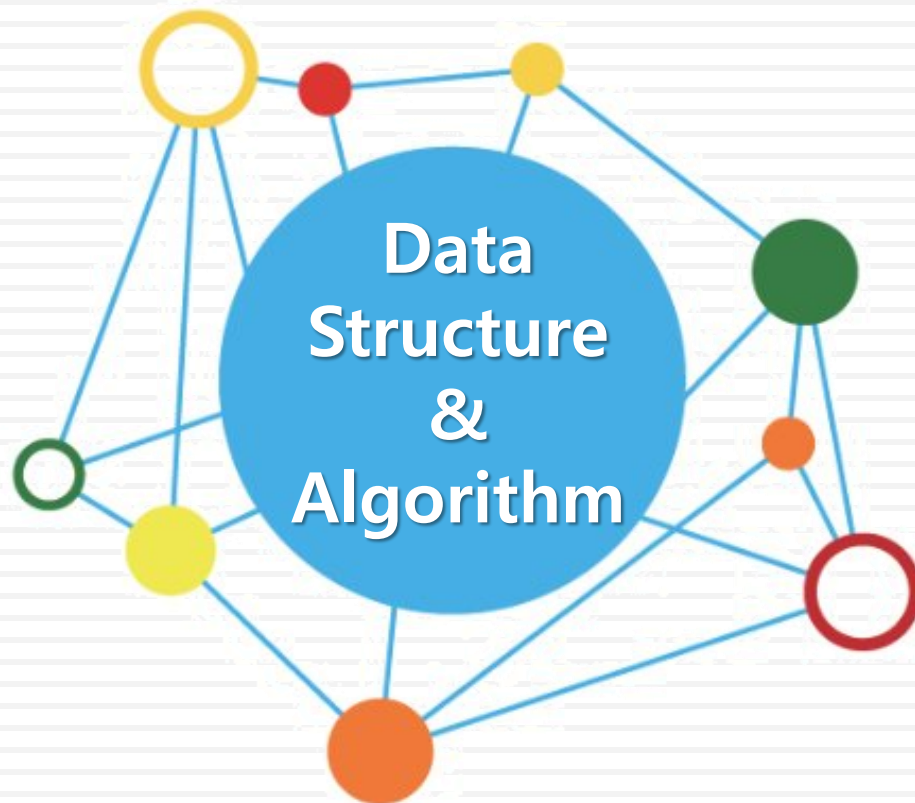


데이터 구조론

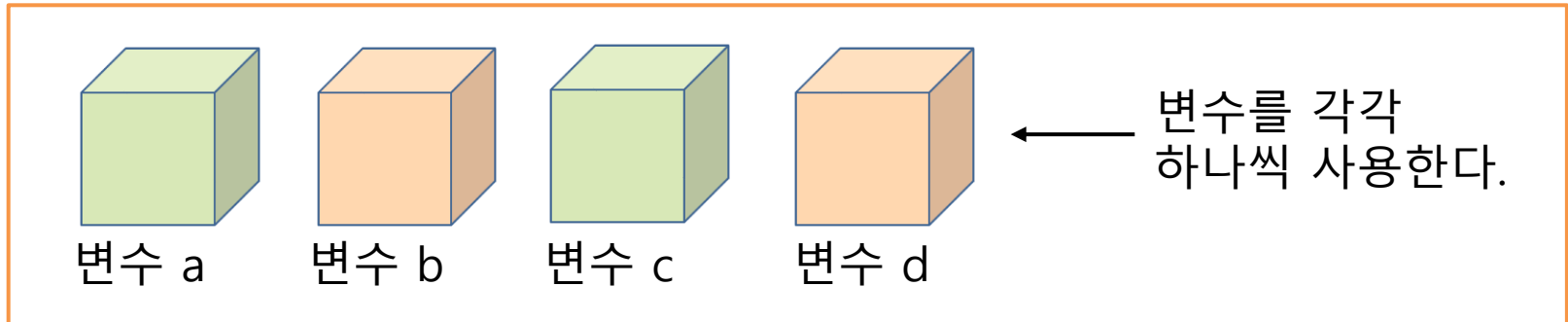


유 길현

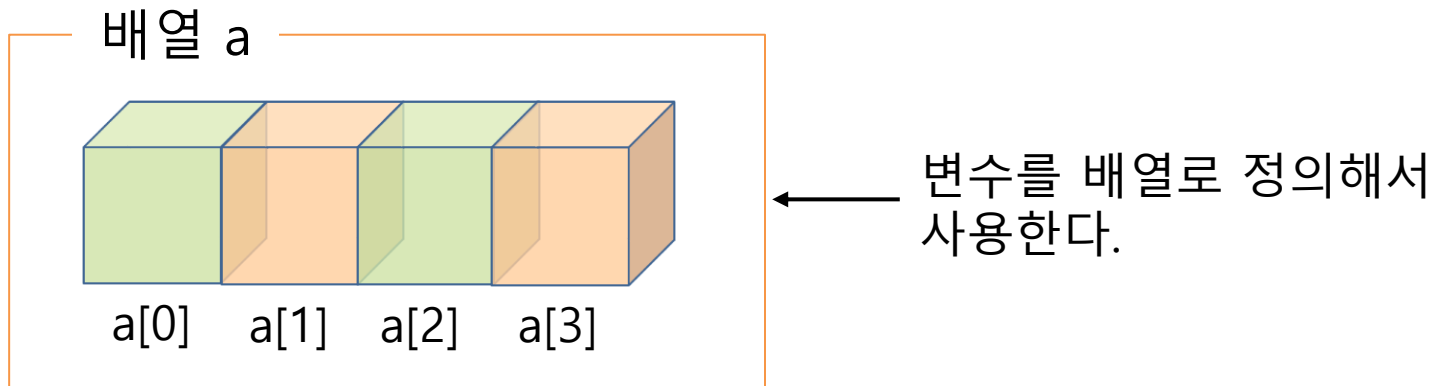
데이터 구조를 위한
C 프로그래밍 기법

❖ 배열(Array)이란 ?

- 동일 자료형을 하나의 이름으로 기억 공간에 저장하는 구조이다.



하나의 이름을 공유하여 자료의 조작이 편리하다.



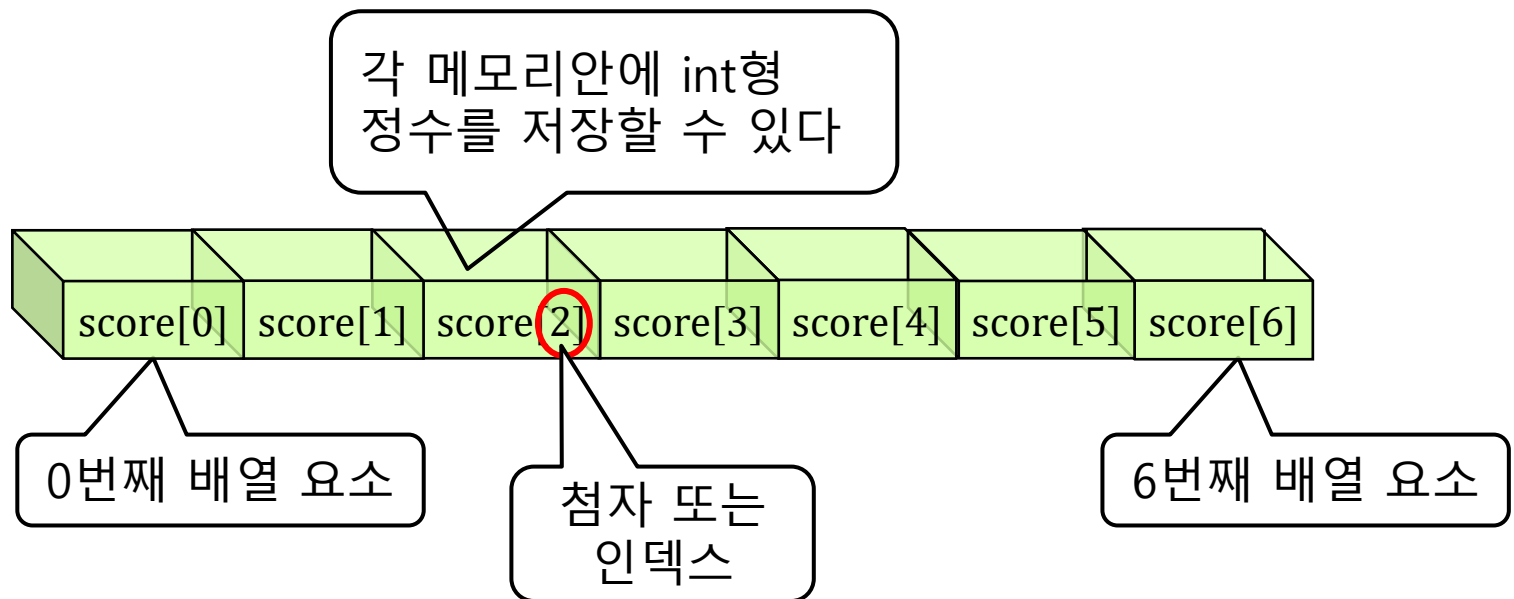
❖ 배열의 선언과 구조

■ 배열 선언

```
int score[7];
```

자료형 배열 이름 배열 크기

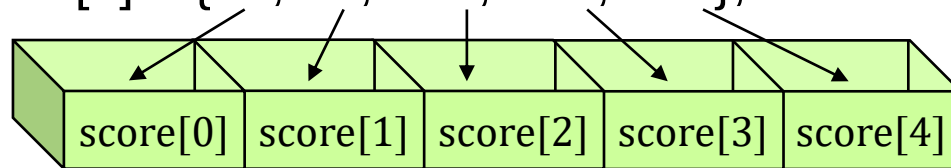
■ 배열 구조



❖ 배열의 초기화

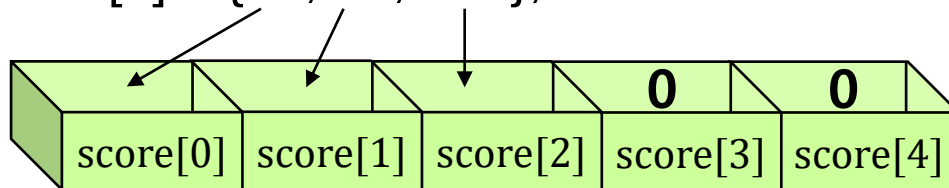
■ 선언과 동시에 초기화

- `int score[5] = {10, 20, 100, 200, 150};`



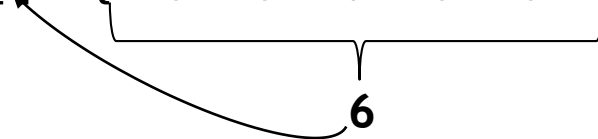
■ 초기값이 부족할 경우 나머지 요소들은 0으로 초기화

- `int score[5] = {10, 20, 100};`



■ 배열의 크기가 주어지지 않는 경우

- `int score[] = {10, 20, 30, 40, 50, 60};`



❖ 배열의 크기 알아내기

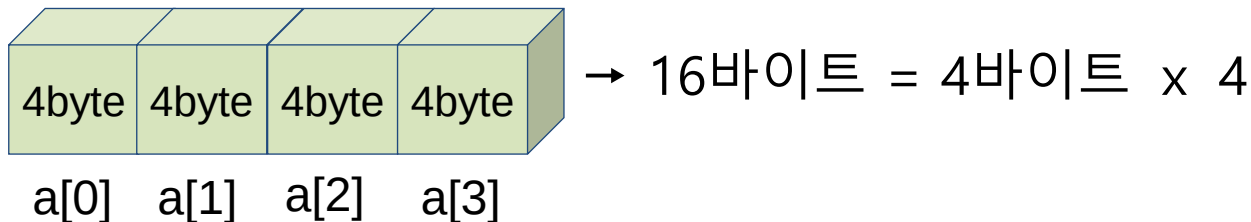
- sizeof() 함수 이용

배열의 크기(요소의 개수) = sizeof(전체 배열 이름) / sizeof(배열의 데이터 형식);

- int a[4]; 배열의 크기 알아내기

배열의 크기(요소의 개수) = sizeof(a) / sizeof(int);

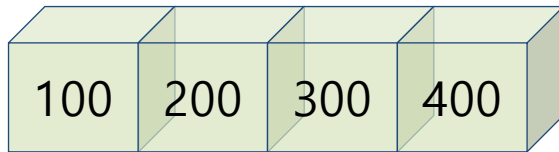
- ① a 배열이 메모리에서 차지하는 크기(4바이트×4개=16바이트)를 알아낸다.
- ② 선언한 배열의 데이터 형식 크기(4바이트)로 나눈다.



❖ 배열과 문자열

- 문자열은 문자의 나열
- 문자열을 구성하는 문자들을 연속적으로 저장하기 위해서는 char형 배열을 사용

정수형 배열

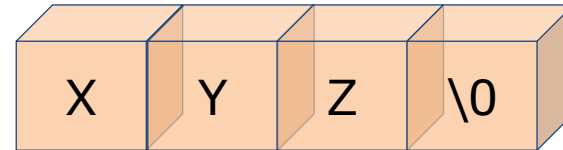


a[0] a[1] a[2] a[3]

정수형 int : 각 4byte

```
int a[4] = {100, 200, 300, 400};
```

문자형 배열



s[0] s[1] s[2] s[3]

문자형 int : 각 1byte

```
char s[4] = {'X', 'Y', 'Z', '\0'};
```

```
char s[4];  
s[0] = 'X';  
s[1] = 'Y';  
s[2] = 'Z';  
s[3] = '\0';
```

```
char s[4] = "XYZ"
```

❖ 1차원 배열

- 가장 기본적인 배열로 다음과 같이 선언 한다.

자료형 배열이름[배열크기]

- 배열 선언과 메모리 할당 구조

```
int score[20];
```

	score[0]	score[1]	score[2]		score[18]	score[19]
score	4byte	4byte	4byte	...	4byte	4byte

❖ 1차원 배열 - 학년별 취득 학점 입출력하기

```
#include <stdio.h>

int main()
{
    int i;

    // 1차원 배열 초기화
    int score[3] = { 91, 86, 97 };
    char grade[3] = { 'A', 'B', 'A' };

    printf("\n *** 학년별 취득 학점 *** \n\n");

    for (i = 0; i<3; i++)
    {
        printf("%3d학년 : 총점 = %d, 등급 = %c\n",
               i + 1, score[i], grade[i]);
    }
    printf("\n");
    return 0;
}
```

❖ 1차원 배열 – 문자 배열에 문자열을 저장하고 출력하기

```
#include <stdio.h>

int main()
{
    char str[20] = "Data Structure! ";
    int i;
    printf("\n문자 배열 str[] : ");
    for (i = 0; str[i]; i++)
    {
        printf("%c", str[i]);
    }
    printf("\n");
    return 0;
}
```

❖ 1차원 배열 – 입력한 문자열의 길이 계산하기

```
#include <stdio.h>

void main()
{
    int i, length = 0;
    char str[50];

    printf("\n문자열을 입력하세요 : ");
    scanf_s("%s", &str, sizeof(str));
    printf("\n입력된 문자열은 \n\n");

    for (i = 0; str[i] != '\0'; i++){
        printf("%c", str[i]);
        length += 1;
    }

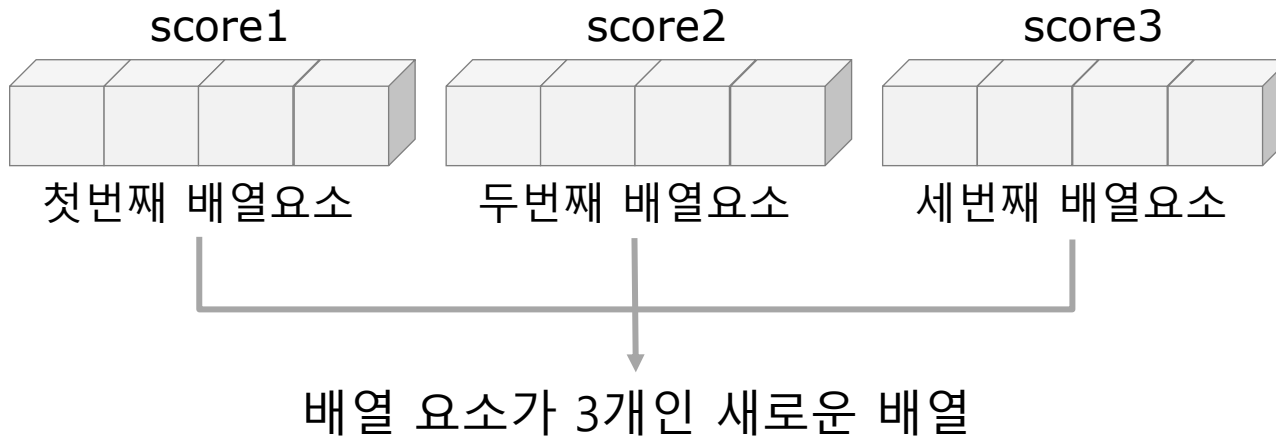
    printf("\n\n입니다.");
    printf("\n\n입력된 문자열의 길이 = %d \n", length);
    getchar();
}
```

❖ 2차원 배열

- 1차원 배열을 배열 요소로 갖는 새로운 배열
- 3명의 학생에 대한 4과목의 학생 점수를 처리하는 예

```
int score1[4];  
int score2[4];  
int score3[4];
```

} 같은 형태의 1차원 배열이 3개 필요



❖ 2차원 배열

- 2차원 배열의 선언

자료형 배열이름 [배열 크기] [배열 크기]

행 개수

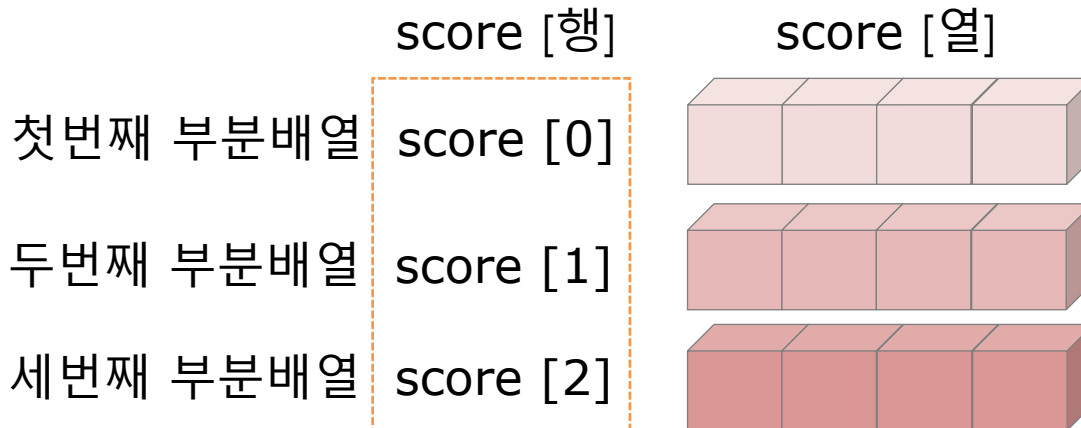
열 개수

↓
행 번호

↓
열 번호

} 논리적 구조에서의 표현

- 2차원 배열은 논리적 행렬로 표현 ⇒ `int score[3][4];`



❖ 2차원 배열

■ 2차원 배열의 초기화

- `int num[3][4]={1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12}`

1	2	3	4
5	6	7	8
9	10	11	12

첫번째 행이 먼저 초기화
된 후에 두 번째, 세 번째
행이 차례로 초기화 된다.

- 초기화 값을 논리적 구조에 맞게 행으로 구분하기 위해 중괄호를 사용한다.

```
int num[3][4] = {{ 1, 2, 3, 4 }, ← num[0]
                  { 5, 6, 7, 8 }, ← num[1]
                  { 9, 10, 11, 12 } }; ← num[2]
```

❖ 2차원 배열 – 학급의 평균 성적과 전체 평균 구하기

```
#include <stdio.h>

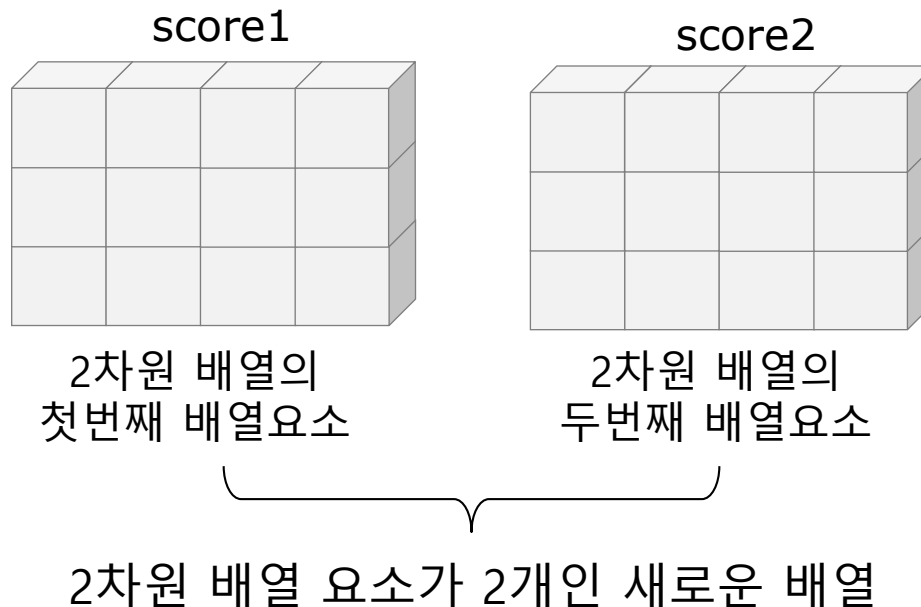
int main()
{
    int s[3][5] = {
        { 0, 1, 2, 3, 4 }, // 첫번째 행의 원소들의 초기값
        { 10, 11, 12, 13, 14 }, // 두번째 행의 원소들의 초기값
        { 20, 21, 22, 23, 24 }, // 세번째 행의 원소들의 초기값
    };

    int clas, student, total, subtotal;
    total = 0;
    for (clas = 0; clas < 3; clas++)
    {
        subtotal = 0;
        for (student = 0; student < 5; student++)
            subtotal += s[clas][student];
        printf("학급 %d의 평균 성적= %d\n", clas+1, subtotal / 3);
        total += subtotal;
    }
    printf("전체 학생들의 평균 성적= %d\n", total / (3 * 5));
    return 0;
}
```

❖ 3차원 배열

- 2차원 배열을 배열 요소로 갖는 새로운 배열
- 학급별 3명의 학생에 대한 4과목의 학생 점수를 처리하는 예

```
int score1[3][4];  
int score2[3][4];
```

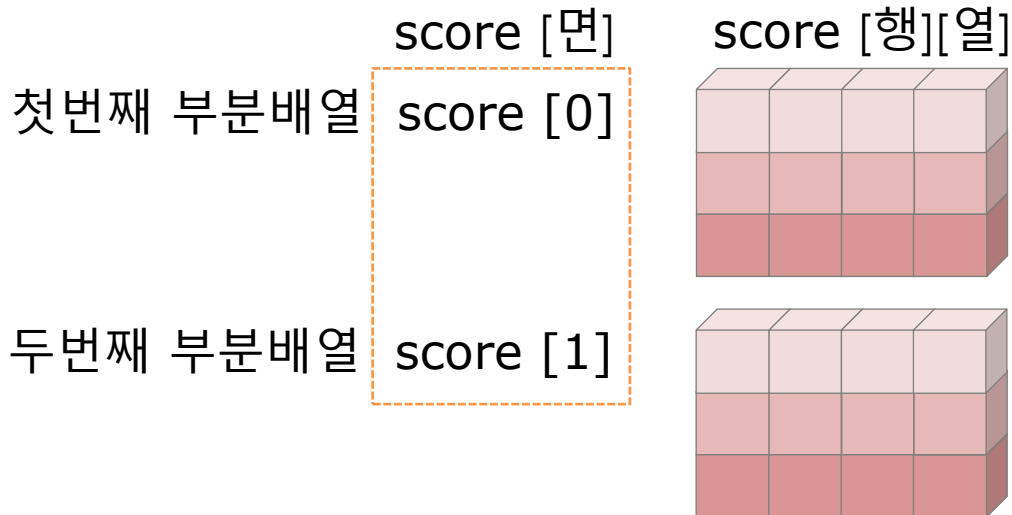
 } 같은 형태의 2차원 배열이 2개 필요

❖ 3차원 배열

- 3차원 배열의 선언 형식

자료형 배열이름 [배열 크기] [배열 크기] [배열 크기]
 면 개수 행 개수 열 개수

- 3차원 배열의 논리적 표현 $\Rightarrow$ `int score[2][3][4];`



❖ 3차원 배열

- 3차원 배열의 초기화는 초기값을 구분하여 지정하거나, 1차원 배열 처럼 순서대로 배열 요소의 초기 값으로 설정

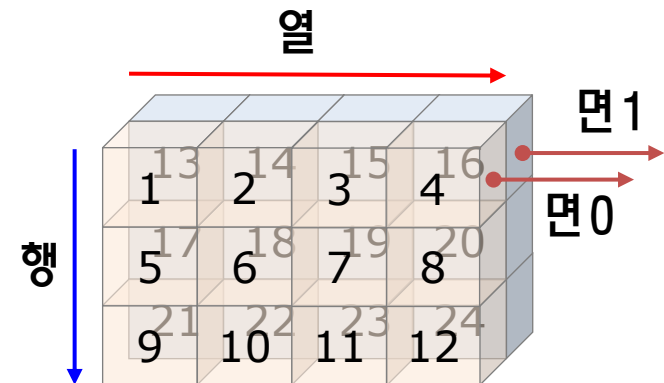
```
int num[2][3][4]={{{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}},  
                  {{13, 14, 15, 16}, {17, 18, 19, 20}, {21, 22, 23, 24}}};
```

면 1

13	14	15	16
17	18	19	20
21	22	23	24

면 0

1	2	3	4
5	6	7	8
9	10	11	12

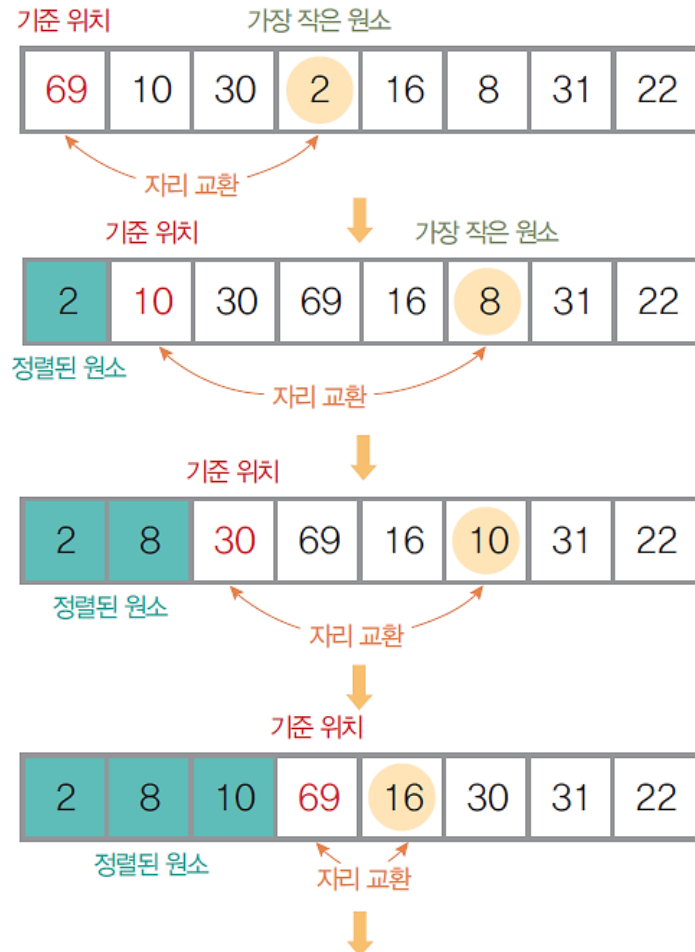


❖ 선택 정렬(Selected Sort)

- 전체 원소들 중에서 기준 위치에 맞는 원소를 선택하여 자리를 교환하는 방식으로 정렬
- 수행방법
 - 전체 원소 중 가장 작은 원소를 찾아 첫 번째 원소와 자리를 교환
 - 두 번째로 작은 원소를 찾아 선택하여 두 번째 원소와 자리를 교환
 - 세 번째로 작은 원소를 찾아 선택하여 세 번째 원소와 자리를 교환
 - 이 과정을 반복하면서 정렬을 완성

❖ 선택 정렬(Selected sort)

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료를 선택 정렬 방법으로 정렬하는 과정



❖ 선택 정렬(Selected sort)

```
#include "stdio.h"

void selectionSort(int a[], int size);

int main()
{
    // 정렬할 초기 원소 배열
    int list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 };
    int i, size = 8;

    // 정렬 전의 원소 출력
    printf("\n정렬할 원소 : ");
    for (i = 0; i < size; i++)
        printf("%d ", a[i]);

    // 선택 정렬 연산 호출
    selectionSort(list, size);
    getchar();
    return 0;
}
```

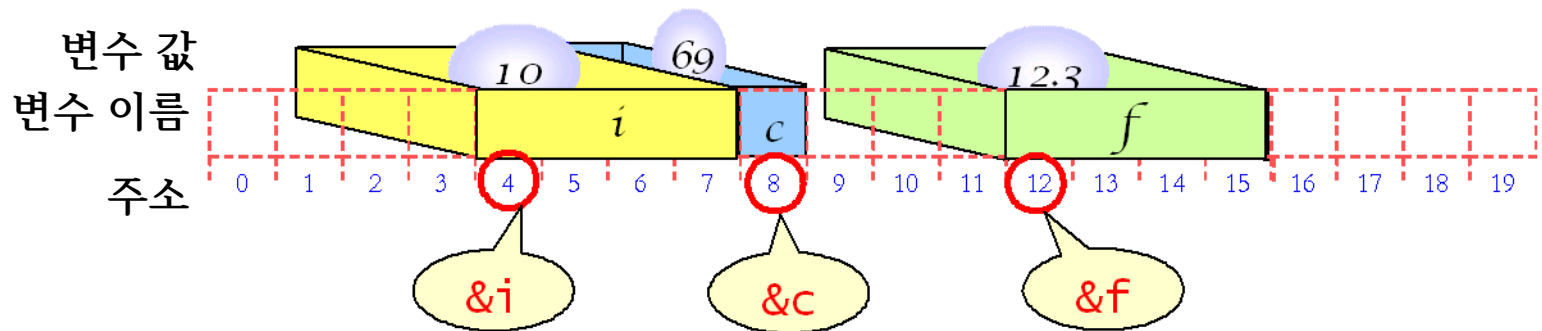
❖ 선택 정렬(Selected sort)

```
void selectionSort(int a[], int size)
{
    int i, j, t, min;
    int temp;

    printf("\n\n<<<<<<<<< 선택 정렬 수행 >>>>>>>>>\n");
    for (i = 0; i < size - 1; i++){
        min = i;
        for (j = i + 1; j < size; j++){
            if (a[j] < a[min])
                min = j;
        }
        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
        printf("\n%d단계 : ", i + 1);
        for (t = 0; t < size; t++)
            printf("%3d ", a[t]);
    }
}
```

❖ 포인터 개념

- 주소 값 : 데이터가 저장된 메모리의 시작 주소를 의미한다.
- 포인터란 메모리의 주소 값을 저장하는 변수이다.
- 연산자
 - 주소 연산자(&) : 해당 변수의 주소 값을 반환
 - 참조 연산자(*) : 포인터변수가 가리키는 주소에 저장된 값을 반환



❖ 포인터 선언

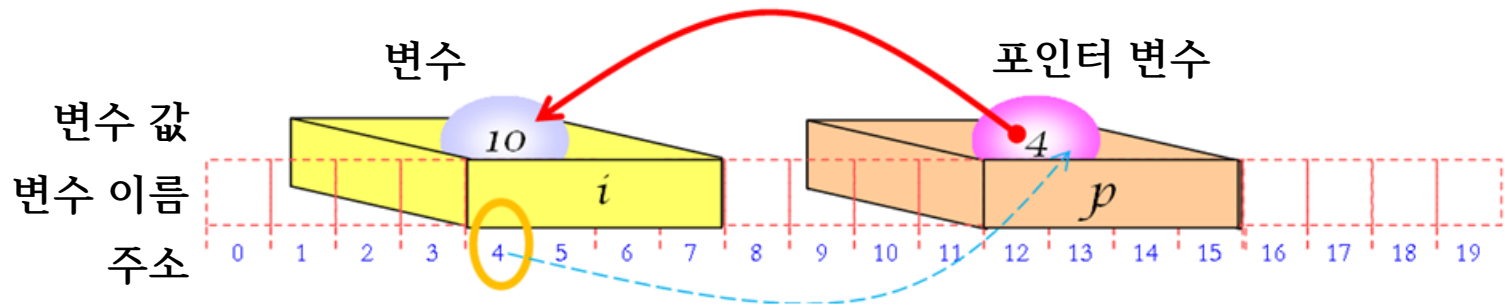
- 포인터는 변수의 주소를 가지고 있는 변수

int *p;

*p가 가리키는 내용은
정수가 된다.

p는 포인터가 된다.

```
int i = 10;    // 정수형 변수 i 선언
int *p;        // 포인터 변수 p 선언
p = &i;        // 변수 i의 주소가 포인터 p로 대입
```

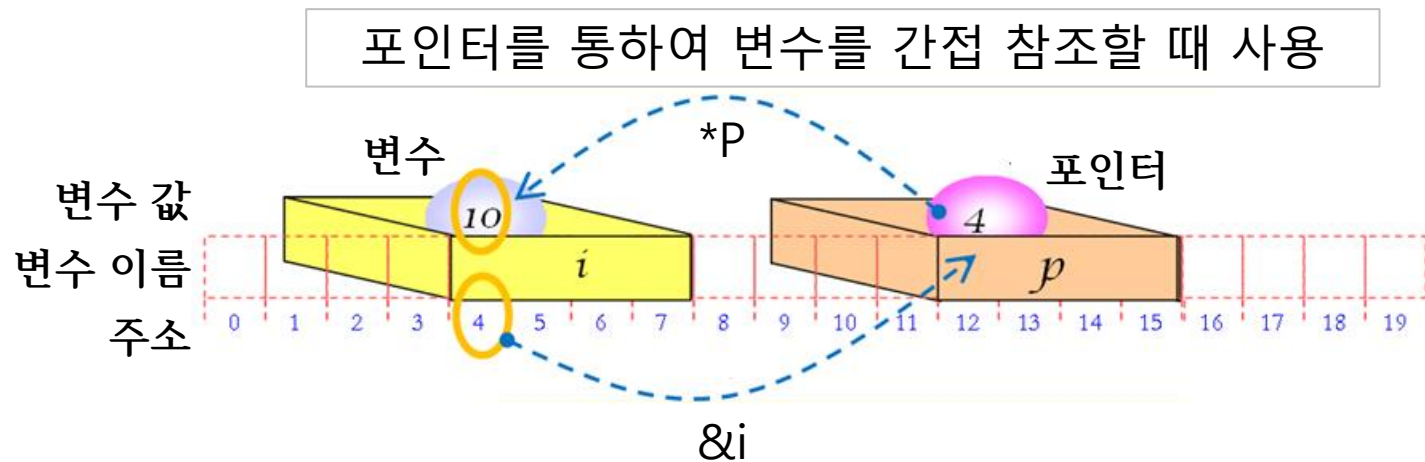


❖ 포인터 선언의 예

포인터 선언	엑세스 범위
char *ptr	ptr에 저장된 메모리 주소로 부터 1 byte의 char 데이터를 엑세스 한다.
short *ptr	ptr에 저장된 메모리 주소로 부터 2 byte의 short 데이터를 엑세스 한다.
int *ptr	ptr에 저장된 메모리 주소로 부터 4 byte의 int 데이터를 엑세스한다.

❖ &연산자와 *연산자

- & 연산자 : 변수의 주소를 반환한다.
- * 연산자 : 포인터가 가리키는 곳의 내용을 반환한다.



변수의 주소를 구하여 포인터에 대입할 때 사용

❖ 포인터 연산

- 포인터는 증가, 감소, 덧셈, 뺄셈 연산이 가능하다.
- 증가 연산의 경우 증가되는 값이 포인터가 가리키는 객체의 크기

포인터 타입	++연산 후 증가되는 값
char	1 byte
short	2 byte
int	4 byte
float	4 byte
double	8 byte

포인터의 증가는 일반 변수와는 달리 가리키는 객체의 크기만큼 증가한다.

❖ 포인터 연산 – 포인터 연산자를 이용해 변수 액세스 하기

```
#include <stdio.h>

int main()
{
    int i = 10, j = 20;
    int *ptr;
    printf("\n i의 값 = %d \n j의 값 = %d", i, j);
    printf("\n i의 메모리 주소(&i) = %u", &i);
    printf("\n j의 메모리 주소(&j) = %u", &j);

    ptr = &i;
    printf("\n\n << ptr=&i 실행 >>");
    printf("\n ptr의 메모리 주소(&ptr) = %u", &ptr);
    printf("\n ptr의 값(ptr) = %u", ptr);
    printf("\n ptr의 참조 값(*ptr) = %d", *ptr);
}
```

❖ 포인터 연산 – 포인터 연산자를 이용해 변수 액세스 하기

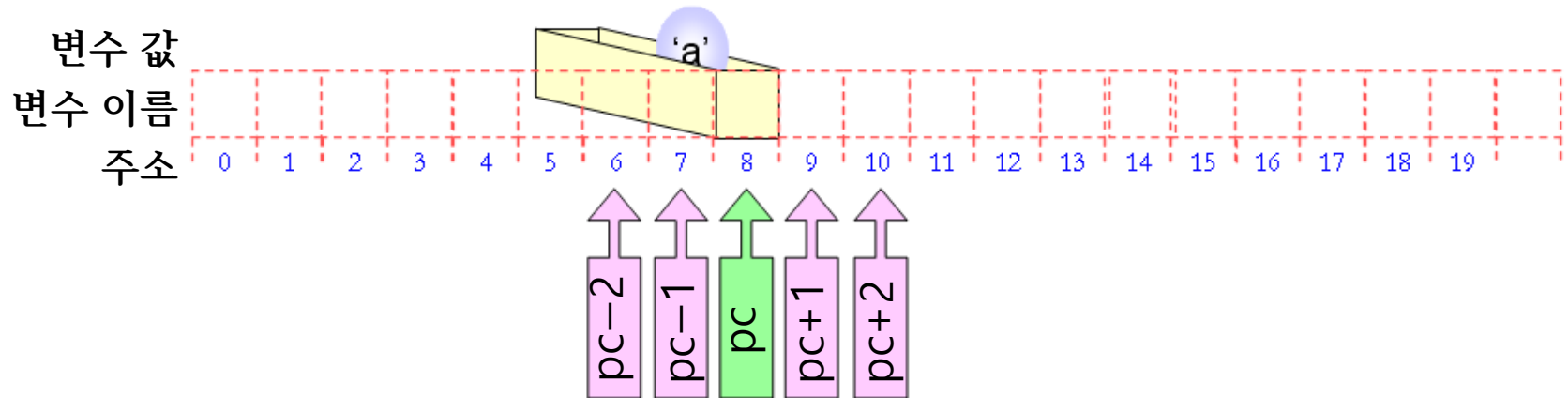
```
ptr = &j;
printf("\n\n << ptr=&j 실행 >>");
printf("\n ptr의 메모리 주소(&ptr) = %u", &ptr);
printf("\n ptr의 값(ptr) = %u", ptr);
printf("\n ptr의 참조값(*ptr) = %d", *ptr);

i = *ptr;
printf("\n\n << i=*ptr 실행 >>");
printf("\n i의 값 = %d", i);

getchar();
return 0;
}
```

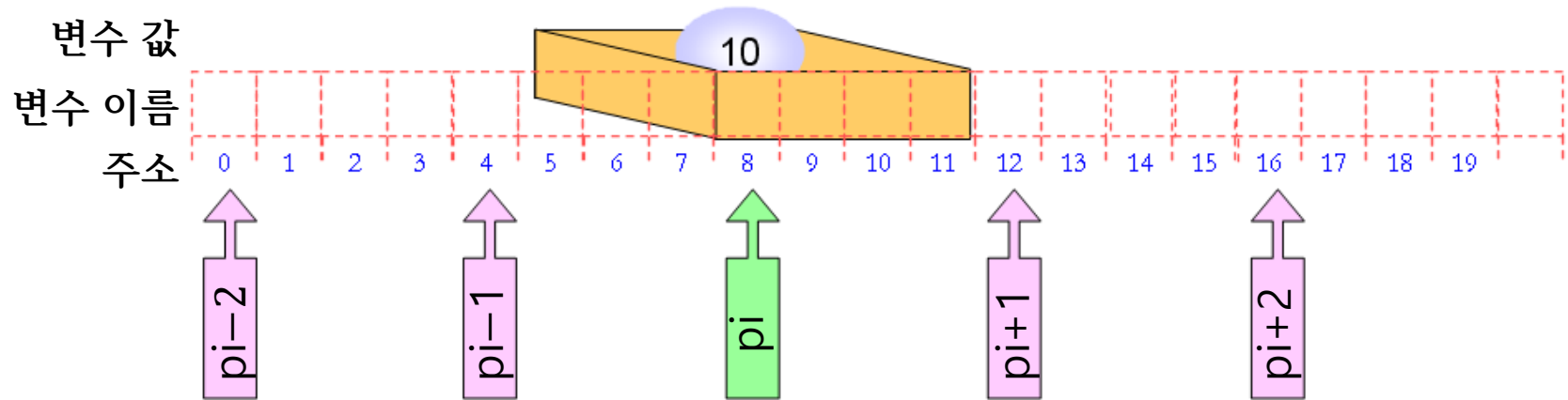
❖ 포인터 증감 연산

```
char c = 'a';  
char *pc;  
pc = &c;
```



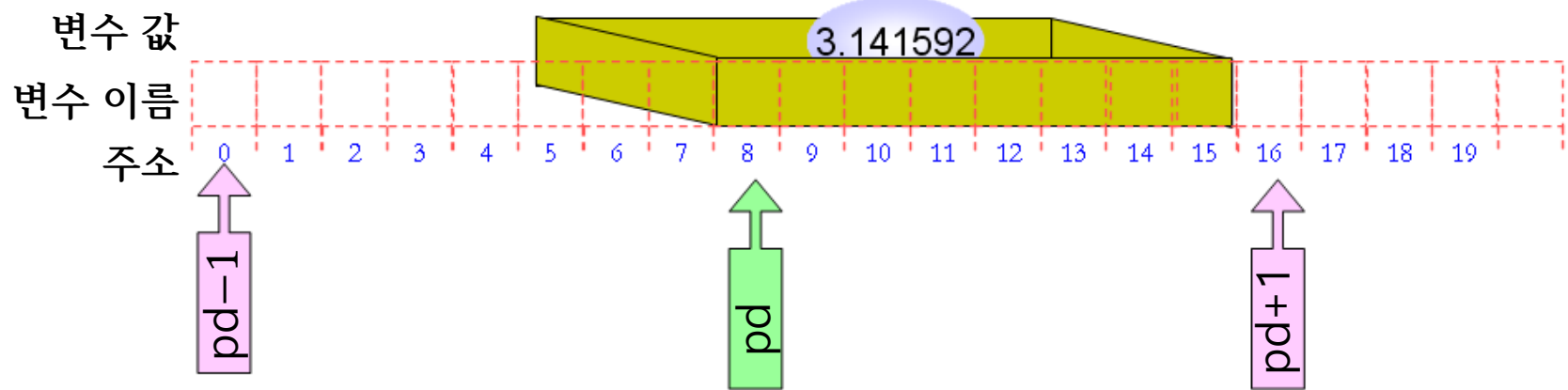
❖ 포인터 증감 연산

```
int i = 10;  
int *pi;  
pi = &i;
```



❖ 포인터 증감 연산

```
double d = 3.141592;  
int *pd;  
pd = &d;
```



❖ 간접 참조 연산자와 증감 연산자

- `*p++;`
 $\Rightarrow$ p가 가리키는 위치에서 값을 가져온 후에 p를 증가한다.
- `(*p)++;`
 $\Rightarrow$ p가 가리키는 위치의 값을 증가한다.

수식	++연산 후 증가되는 값
<code>v = *p++</code>	p가 가리키는 값을 v에 대입한 후에 p를 증가한다.
<code>v = (*p)++</code>	p가 가리키는 값을 v에 대입한 후에 가리키는 값을 증가한다.
<code>v = *++p</code>	p를 증가시킨 후에 p가 가리키는 값을 v에 대입한다.
<code>v = ++*p</code>	p가 가리키는 값을 가져온 후에 그 값을 증가하여 v에 대입한다.

❖ 포인터 증감 연산 예제

```
#include <stdio.h>
int main()
{
    int v = 0, i = 10;
    int *pi = &i;

    v = *pi;
    printf("v = %d i = %d pi = %d\n", v, i, pi);
    v = (*pi)++;
    printf("v = %d i = %d pi = %d\n", v, i, pi);
    return 0;
}
```

```
#include <stdio.h>
int main()
{
    int v = 0, i = 10;
    int *pi = &i;

    v = *pi;
    printf("v = %d i = %d pi = %d\n", v, i, pi);
    v = *pi++;
    printf("v = %d i = %d pi = %d\n", v, i, pi);
    return 0;
}
```

❖ 포인터 증감 연산 예제

```
#include <stdio.h>
int main()
{
    int v = 0, i = 10;
    int *pi = &i;

    v = *pi;
    printf("v = %d i = %d pi = %d\n", v, i, pi);
    v = *++pi;
    printf("v = %d i = %d pi = %d\n", v, i, pi);
    return 0;
}
```

```
#include <stdio.h>
int main()
{
    int v = 0, i = 10;
    int *pi = &i;

    v = *pi;
    printf("v = %d i = %d pi = %d\n", v, i, pi);
    v = ++*pi;
    printf("v = %d i = %d pi = %d\n", v, i, pi);
    return 0;
}
```

❖ 포인터 배열

- 포인터 자료형을 배열로 구성
 - 여러 개의 포인터를 하나의 배열로 구성한 배열의 특징과 포인터의 특징을 모두 활용할 수 있다.
- 포인터 배열의 선언 형식

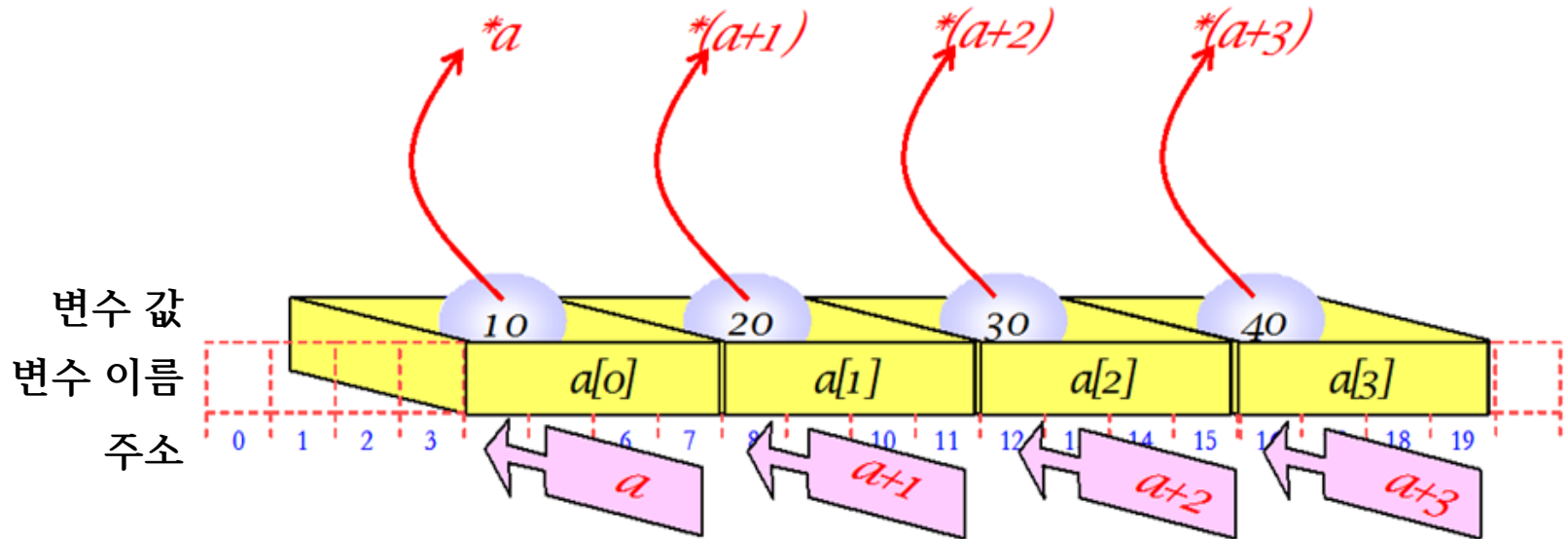
int *ap[10];

자료형 배열크기

포인터 배열 이름

❖ 포인터 배열

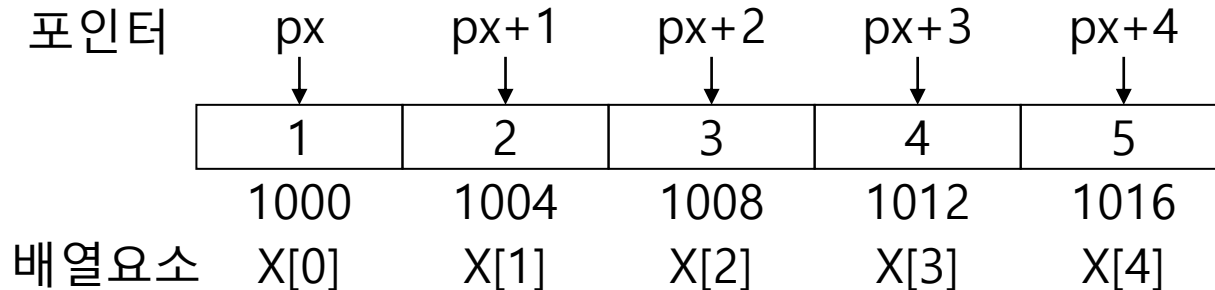
- 인덱스 표기법을 포인터에 사용할 수 있다.



❖ 배열연산과 포인터 연산

- 배열연산 : 배열의 첨자를 사용해 배열 원소를 참조
- 포인터 연산 : 포인터가 배열을 가리키고 포인터에 관계된 연산을 수행

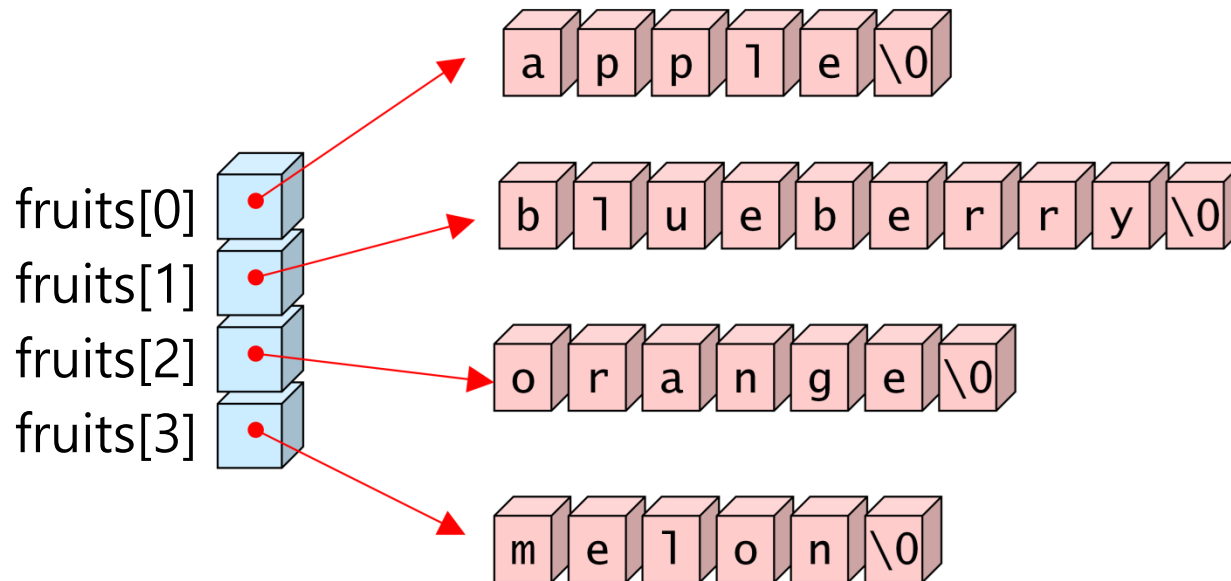
```
int x[5] = { 1, 2, 3, 4, 5 };  
int *px;  
px = x;      // *px = &x[0]와 같다.  
              // 포인터 px는 현재 배열의 시작 주소
```



❖ 포인터 배열

■ 문자형 포인터 배열

```
char *fruits[ ] = {  
    "apple",  
    "blueberry",  
    "orange",  
    "melon"  
};
```



❖ 문자형 포인터 배열

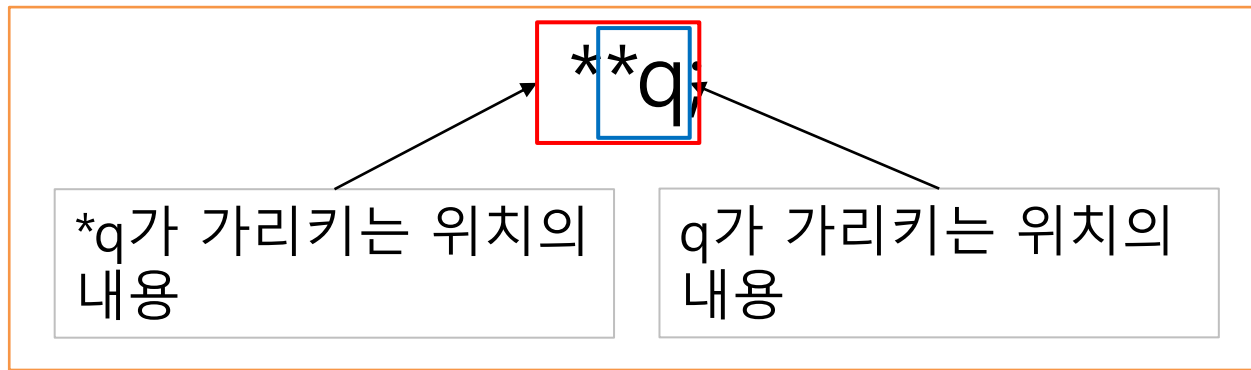
```
#include <stdio.h>

int main()
{
    int i;
    char *fruits[]{"apple",
                  "blueberry",
                  "orange",
                  "melon"};

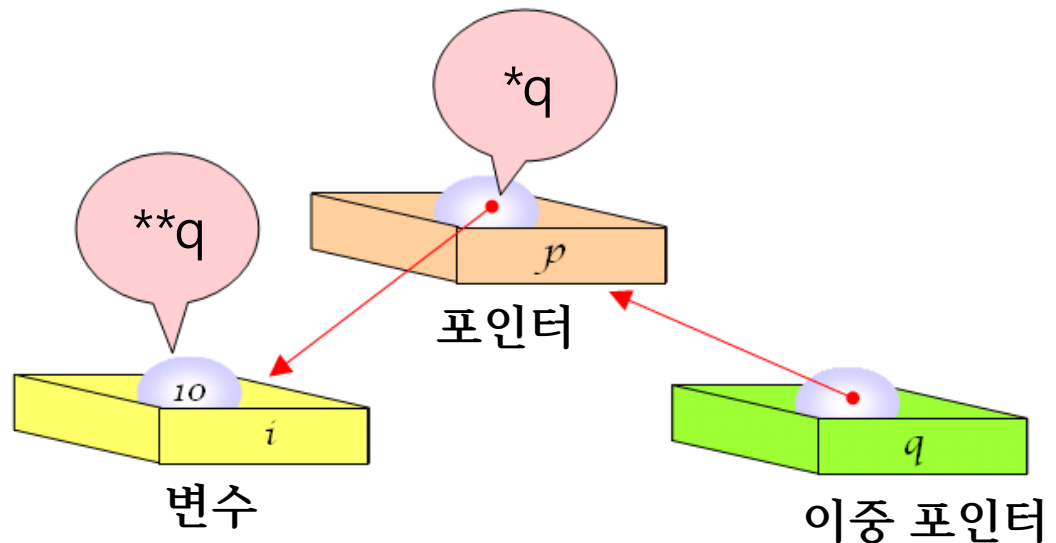
    for(i=0; i<4; i++)
    {
        printf("fruits[%d] -> %s\n", i, fruits[i]);
    }
    return 0;
}
```

❖ 이중 포인터

- 포인터를 가리키고 있는 포인터



```
int i = 10;  
int *p = &i;  
int **q = &p;
```



❖ 이중 포인터 예제

```
#include <stdio.h>

int main()
{
    int i = 100;
    int *p = &i;
    int **q = &p;

    printf("i = %d   *p = %d   **q = %d \n", i, *p, **q);

    *p = 200;
    printf("i = %d   *p = %d   **q = %d \n", i, *p, **q);

    **q = 300;
    printf("i = %d   *p = %d   **q = %d \n", i, *p, **q);

    return 0;
}
```

❖ 선택 정렬(Selected sort)

```
#include "stdio.h"

void SelectionSort(int *a, int size);

int main()
{
    // 정렬할 초기 원소 배열
    int list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 };
    int i, size = 8;
    // 정렬 전의 원소 출력
    printf("\n정렬할 원소 : ");
    for (i = 0; i < size; i++)
        printf("%d ", list[i]);

    // 선택 정렬 연산 호출
    SelectionSort(list, size);
    getchar();
    return 0;
}
```

❖ 선택 정렬(Selected sort)

```
// 선택 정렬하는 연산
void SelectionSort(int *a, int size)
{
    int i, j, t, min, temp;

    printf("\n\n<<<<<<<<< 선택 정렬 수행 >>>>>>>>>\n");
    for (i = 0; i < size - 1; i++){
        min = i;
        for (j = i + 1; j < size; j++){
            if (*(a+j) < *(a+min))
                min = j;
        }
        temp = *(a+i);
        *(a+i) = *(a+min);
        *(a+min) = temp;
        printf("\n%d단계 : ", i + 1);
        for (t = 0; t < size; t++)
            printf("%3d ", *(a+t));
    }
}
```

❖ 구조체 개념

- 구조체란 C언어의 기본 자료형을 가지고 새롭게 정의할 수 있는 사용자 정의 자료형 이다.
- 구조체는 하나의 이름공간에 서로 다른 자료형을 묶어 통합된 변수로 사용한다.
- 여러 자료형의 필드를 가지고 있는 레코드를 만들 때 사용

김선달	2019년 입사	3500	
이몽룡	2020년 입사	3000	← 레코드
홍길동	2017년 입사	4500	
박문수	2018년 입사	4000	

↑
필드

❖ 구조체 선언

- 구조체 선언이란 구조체를 만드는 작업으로 struct 명령어를 사용하여 구조체 변수를 선언하는 것을 의미한다.

이름(태그)

```
struct employee{    // 종업원 구조체 선언
    char name[10];
    int year;
    int pay;
}; // 구조체 선언의 끝을 나타내는 세미콜론(;)은 반드시 붙여야 함
```

구조체 멤버

❖ 구조체 변수의 선언 방법

방법	예
구조체형을 선언한 후에 구조체 변수 선언	<pre>struct employee { char name[10]; int year; int pay; }; struct employee Lee;</pre>
구조체형과 구조체 변수를 연결하여 선언	<pre>struct employee { char name[10]; int year; int pay; } Lee;</pre>
구조체형 이름을 생략하고 구조체 변수 이름만 선언	<pre>struct { char name[10]; int year; int pay; } Lee;</pre>

❖ 구조체 변수의 초기화

- 일반 변수 초기화와 마찬가지로 구조체 변수 초기화하려면 구조체 변수를 선언하면서 변수의 초기값 지정
- 일반 변수는 값을 하나만 가지므로 초기값도 하나지만, 구조체는 내부 항목이 여러 개일 수 있으므로 내부 항목의 자료형과 개수를 순서에 맞추어 초기값 리스트로 지정하고 중괄호({ }) 사용
- 구조체 초기화

```
struct employee { // 구조체 선언
    char name[10];
    int year;
    int pay;
} 구조체 멤버
};
struct smployee Lee = {"Ann", 2015, 4200}; // 구조체 변수의 초기화
```

❖ 구조체 멤버 변수의 의 참조

- 구조체 멤버를 참조 하려면 . 연산자를 사용한다.

```
struct employee Lee;  
Lee.name = "Ann"  
Lee.year = 2015;  
Lee.pay = 4200;
```

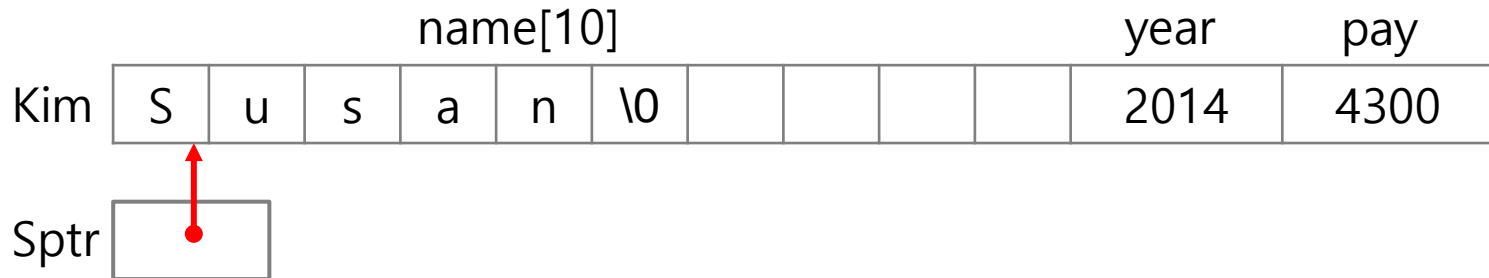
.

기호는 구조체에서 멤버를 참조할 때 사용하는 연산자

❖ 구조체 멤버 참조

- 구조체 포인터에서 구조체 멤버를 참조할 때 -> 연산자를 사용한다.

```
struct employee Kim;  
struct employee *Sptr = &Kim;  
Sptr->name = "Susan"  
Sptr->year = 2014;  
Sptr->pay = 4300;
```



`(*Sptr).name` == `p->name`

❖ 구조체 예제

```
#include <stdio.h>
#include <string.h>

struct employee {
    char name[10];
    int year;
    int pay;
};

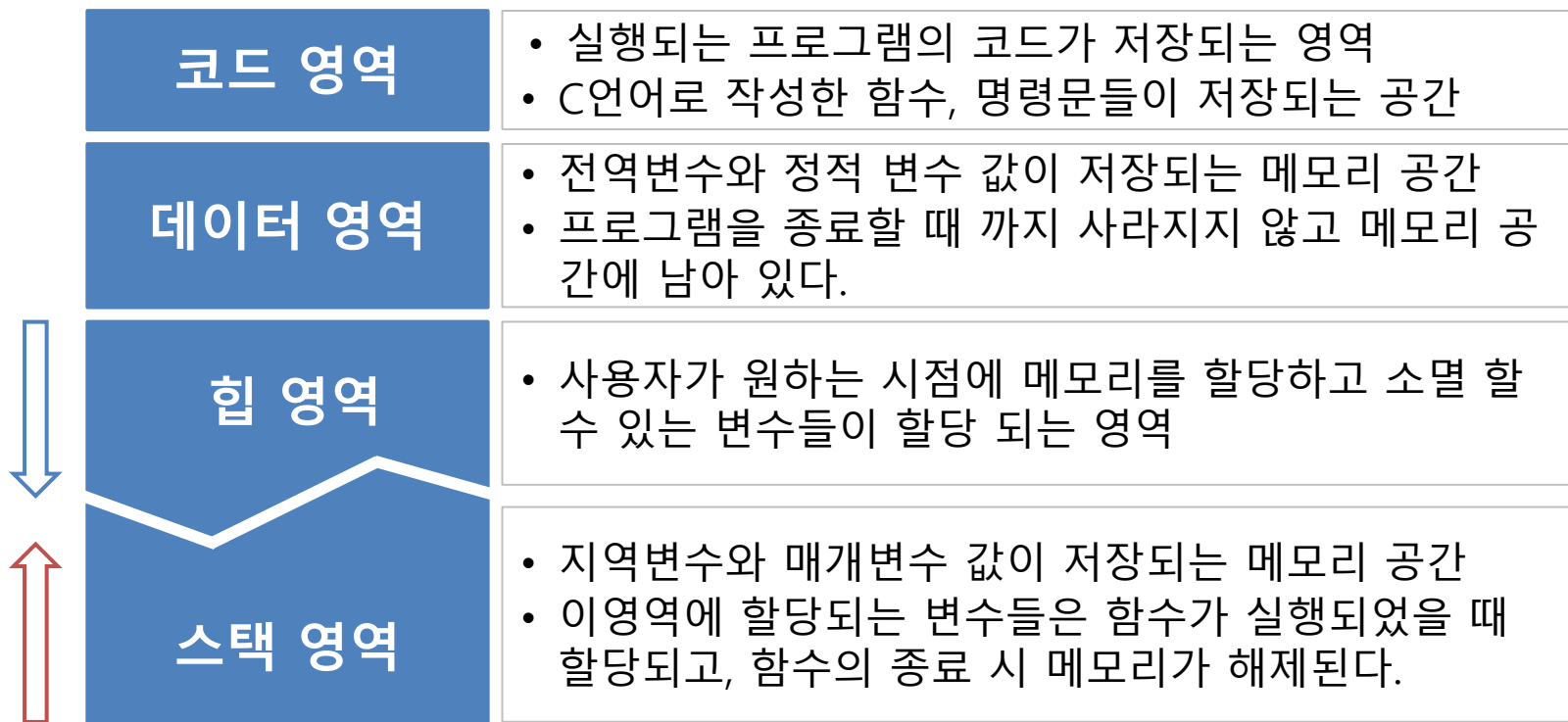
int main()
{
    struct employee Lee;
    struct employee *Sptr = &Lee;
    strcpy(Sptr->name, "홍길동");
    Sptr->year = 2015;
    Sptr->pay = 5900;

    printf("\n 이름 : %s", Sptr->name);
    printf("\n 입사 : %d", Sptr->year);
    printf("\n 연봉 : %d", Sptr->pay);
    printf("\n");
    return 0;
}
```

❖ C언어의 메모리 구조

- 프로그램 실행 시 운영체제는 실행 프로그램을 위해 메모리 공간을 할당 하며 메모리 공간은 코드(code), 데이터(data), 힙(heap), 스택(stack) 영역으로 나뉘어진다.

운영체제에 의해 할당된 메모리 공간



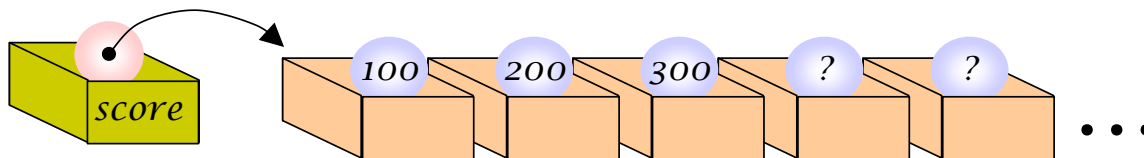
❖ 동적 메모리 할당

- 실행 도중에 동적으로 메모리를 할당받는 것
- 사용이 끝나면 시스템에 메모리를 반납
- 필요한 만큼 할당을 받아 메모리를 효율적으로 사용
- `void *malloc(size_t size)`
 - size는 바이트수
 - `malloc()` 함수는 메모리 블록의 첫 번째 바이트에 대한 주소를 반환
 - 요청한 메모리 공간을 할당할 수 없는 경우에 NULL값을 반환

```
int *score;
score = (int *)malloc(100*sizeof(int));
if( score == NULL ){
    ... // 오류 처리
}
```

❖ 동적 메모리 사용

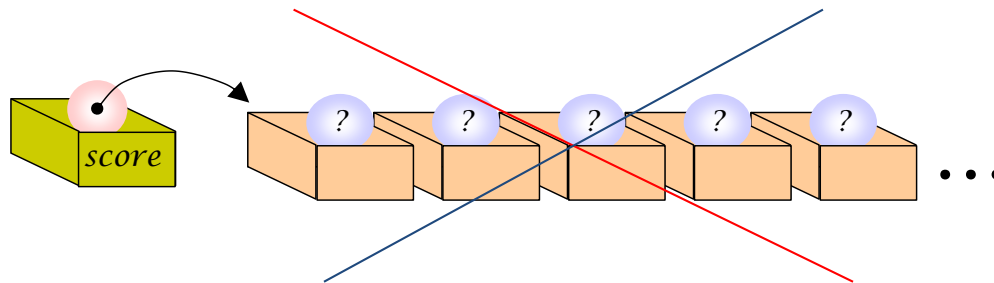
- 첫 번째 방법 : 포인터를 사용
 - `*score = 100;`
 - `*(score+1) = 200;`
 - `*(score+2) = 300;`
 - ...
- 두 번째 방법 : 동적 메모리를 배열과 같이 취급
 - `score[0] = 100;`
 - `score[1] = 200;`
 - `score[2] = 300;`
 - ...



❖ 동적 메모리 반납

- `void free(void *ptr)`
 - `free()`는 동적으로 할당되었던 메모리 블록을 시스템에 반납
 - `ptr`은 `malloc()`을 이용하여 할당된 메모리를 가리키는 포인터

```
int *score;  
score = (int *)malloc(100*sizeof(int));  
...  
free(score);
```



❖ 예제

```
#include "stdio.h"
#include "stdlib.h"
#include "string.h"

struct Book {
    int number;
    char title[20];
};

int main()
{
    struct Book *p;

    p = (struct Book *)malloc(2 * sizeof(struct Book));

    if (p == NULL) {
        printf("메모리 할당 오류\n");
        exit(1);
    }
}
```

❖ 예제

```
p->number = 1;
strcpy(p->title, "C Programming");

(p + 1)->number = 2;
strcpy((p + 1)->title, "Data Structure");

printf("%d %s\n", p->number, p->title);
printf("%d %s\n", (p+1)->number, (p+1)->title);

free(p);
printf("=====메모리할당 해제=====\n");
printf("%d %s\n", p->number, p->title);
printf("%d %s\n", (p + 1)->number, (p + 1)->title);

return 0;
}
```

❖ 입력된 문자를 알파벳 오름차순으로 정렬

알파벳 대문자의와 숫자(0~9)로만 구성된 문자열이 입력으로 주어 집니다. 이때 모든 알파벳을 오름차순으로 정렬하여 출력 하고 이어서 모든 숫자를 더한 값을 출력 합니다. 예를 들어 KIKA5CB7 이라는 값이 들어오면 ABCKK13을 출력 합니다.

- 입력 조건 : 첫째 줄에 하나의 문자열 S 가 주어집니다. ($1 \leq S$ 의 길이 ≤ 20)
- 출력 조건 : 첫째 줄에 문제에서 요구하는 정답을 출력 합니다.

입력 예시 1

KIKA5CB7

출력 예시 1

ABCKK13

입력 예시 2

AJKDLSI412K4JSJ9D

출력 예시 2

ADDIJJJKKLSS20